

GRIMOIRE VAULT · V1.0.0

История проекта

*История проекта — как создавалось,
технологии, аналоги*

Grimoire Vault — история проекта

Как это создавалось, какие решения принимались, что внутри стека и почему именно так. Документ на тех, кто хочет понять не только «что есть», но и «почему так».

Часть I · Начало

Замысел

Идея проста: **личная база знаний, которая работает с любого устройства, ничего не теряет и не запирает в себе данные.**

Аналогов на рынке много — Obsidian, Notion, Logseq, Roam, Anytype, Pocket, Raindrop, Evernote. У каждого своя проблема:

АНАЛОГ	ЧТО НЕ НРАВИТСЯ
Notion	Закрытый формат, медленный, нельзя без интернета, сложный экспорт
Obsidian	Локальные .md-файлы — синхронизация платная, мобильное приложение хуже десктопа
Logseq / Roam	Outlining-парадигма не для всех, мобильное использование клин
Pocket	Только статьи, не файлы, не код, не идеи; куплен Mozilla, неясное будущее
Raindrop	Только закладки; без AI, поиск по словам
Evernote	Тяжёлое, бесплатный tier режут год за годом
Anytype	Хорошая идея, но вес десктопа + сырой мобильный клиент

Что хотелось:

- Жёсткая структура из 13 категорий (Documents, Web, YouTube, Designs, Ideas, ...) — как ящики в старом библиотечном шкафу.
- Telegram-бот как основной capture: переслал ссылку → она в нужной категории через 2 секунды.
- Поиск, который понимает смысл, а не только слова.
- Зашифрованный vault для паролей рядом с заметками.
- Web-first, с PWA для телефона — без отдельных мобильных приложений.
- Полное владение данными: экспорт в JSON / ZIP, импорт обратно.
- Работа в одиночку, но с возможностью пригласить семью на отдельные «комнаты» (shared vaults).

Эволюция стека

Изначально это был статический HTML-прототип в браузере с `sessionStorage`. Чтобы превратить его в **production-grade multi-device app**, нужно было выбрать стек. Вот критерии:

1. **Бесплатный** на старте (free tier должен покрывать персональное использование на годы вперёд).
2. **Мульти-устройство** без своего бэкенда — авторизация и БД через managed-сервис.
3. **TypeScript end-to-end** для безопасности рефакторинга.
4. **RSC-friendly** для быстрого первого пейнта и offline-PWA.
5. **Без vendor lock-in** — экспорт в стандартный JSON, миграция за минуту.

Получился стек, описанный ниже.

Часть II · Технологии стека

Next.js 16 — фронтенд + API в одном

Next.js — фреймворк от Vercel поверх React.

Зачем:

- **App Router + Server Components** — каждый компонент по умолчанию рендерится на сервере. Не нужно отдельное API для базовых запросов — данные тянутся прямо в JSX. Это сильно сокращает JS-bundle (он не передаётся клиенту вообще для server-only страниц).
- **Streaming через Suspense** — главная страница начинает рендериться ещё до того, как Postgres ответил на запрос счётчиков по категориям. Первая отрисовка — за ~150 ms независимо от database latency.
- **Turbopack** — новый сборщик, в разы быстрее Webpack. Дев-сервер стартует за 2 секунды, hot reload — за 100 ms.
- **One repo, one runtime** — фронтенд и API живут в одном проекте. `app/api/*` — это полноценные route handlers. Никакой межпроектной синхронизации типов.

Простому юзеру не нужно ничего знать про Next.js. Это деталь реализации. Браузеру важно, что страница быстро открывается.

React 19 + Tailwind v4

React 19 — последняя стабильная. Принципиальное здесь: **серверные компоненты как default**. Только то, что нужно интерактивность, явно маркируется `"use client"`. Это фактически противоположно традиционному React.

Tailwind v4 — utility-first CSS. Никакого CSS-in-JS, никаких `.module.css`. Дизайн-система живёт в `app/globals.css` (24 цвета + 4 семейства шрифтов: Fraunces для display, Manrope для UI, JetBrains Mono для технических меток).

Почему не shadcn/ui или Mantine — потому что весь UI кастомный под «галантерейный» дизайн (ivory + emerald + золотая обводка). Готовые компоненты пришлось бы переписывать целиком.

Что мы используем:

- **Postgres 17.4** — основная БД. Все 8 таблиц + индексы + RPCs + RLS-политики живут в нём.
- **Auth** — email magic-link + password. JWT-токены.
- **RLS (Row-Level Security)** — авторизация **на уровне БД, не на уровне приложения**. Каждый SELECT / INSERT / UPDATE / DELETE фильтруется политиками `auth.uid()`-based. Это значит: даже если кто-то получит доступ к anon-ключу и попытается напрямую дёрнуть PostgREST — он не сможет прочитать чужие записи.
- **Realtime** — websocket-канал, который пушит INSERT/UPDATE/DELETE события в браузер. Подписался на `entries:misc` — увидел live, как бот добавил новую запись из другого устройства.
- **pgvector** — расширение Postgres для cosine-similarity поиска. Используется HNSW-индекс с 384-мерными векторами.

Почему Supabase, а не своё: один сервис заменяет три (Auth + Postgres + Realtime). Free tier — 500 MB БД, 2 GB трафика, неограниченное число API-запросов. Этого хватит лет на пять личного использования.

Что важно знать про PostgreSQL для понимания проекта:

1. **Реляционная модель:** данные живут в таблицах со столбцами и строками, связанными через `FOREIGN KEY`.
2. **Транзакции:** либо все изменения коммитятся, либо ни одно.
3. **Индексы:** ускоряют поиск. В нашем проекте: HNSW для cosine, GIN для tsvector, partial indexes для inbox-фильтра.
4. **Триггеры:** функции, автоматически срабатывающие на INSERT / UPDATE. Используются для maintain `search_tsv`, `triaged_at` defaults, vault membership seeding.
5. **RLS:** «эта строка видна только если выполняется условие `auth.uid() = user_id`». Защита данных на уровне самой БД.

Если первый раз видишь Postgres — почитай первую главу [The Internals of PostgreSQL](#) или [Use The Index, Luke](#) для фундамента. Этого хватит на 90% работы с проектом.

Cloudflare R2 — хранилище файлов

R2 — S3-совместимое объектное хранилище. Главная фишка — **\$0 egress** (S3 берёт деньги за каждый скачанный байт; R2 не берёт).

Что в нём лежит:

- `users/<uid>/originals/<uuid>.<ext>` — оригиналы документов / видео.
- `users/<uid>/covers/<uuid>.webp` — обложки (4:3).
- `users/<uid>/thumbs/<uuid>.webp` — превью видео (16:9).

Загрузка — через **presigned URL**: клиент просит у сервера короткоживущую ссылку, кладёт файл прямо в R2 без участия Vercel-функции. Это значит, что upload 100MB-файла **не нагружает наш сервер**.

Скачивание — через signed proxy `/api/r2/object/[...key]`, который проверяет, что user.id владеет ключом, и стримит байты из R2.

GRIMOIRE VAULT · PROJECT-STORY

Бакет приватный. Никто не может скачать файл, не пройдя через нас.

WebP-транскодинг в браузере — перед PUT'ом в R2 любой JPEG/PNG автоматически конвертится в WebP с качеством 85% через `<canvas>.toBlob`. Это экономит 30-60% трафика на каждом скачивании, без серверного CPU.

`@huggingface/transformers` — **embeddings в браузере**

Семантический поиск работает через 384-мерные эмбединги — векторы, кодирующие смысл текста. Близкие по смыслу тексты → близкие векторы.

Где они считаются: в браузере, через ONNX-модель `multilingual-e5-small` (118 MB, q8-квантизованная версия — ~30 MB).

Почему не на сервере:

- Vercel functions имеют 1 GB памяти и cold start ~1 секунду. Загрузить туда 100+ MB модели — драматичный cold start на каждом редком запросе.
- На каждый embed Vercel брал бы compute time. Браузер делает то же бесплатно.
- **Privacy:** текст пользователя для эмбединга не уходит на стороннее API.

Когда модель скачивается:

- При первом переключении в режим «Гибрид» или «По смыслу» в `/search`.
- При первом нажатии «Запустить бэкфилл» в Settings.

После скачивания — кэшируется в **IndexedDB** браузера. Дальше работает офлайн и мгновенно.

Альтернативы, которые отбросили: Voyage AI (платный API), OpenAI ada-002 (платный), Supabase Edge Functions с gte-small (бесплатный, но англоязычный + cold start).

grammY — Telegram-бот

grammY — TypeScript-фреймворк для Telegram-ботов.

В одном файле `lib/telegram/bot.ts`:

- `/start`, `/help`, `/link <code>`, `/unlink`, `/search <q>` команды.
- `bot.on("message:text")` — текстовые сообщения; URL-парсер определяет YouTube vs обычный URL.
- `bot.on("message:photo")` — фото-сообщения.

Деплой через webhook: Telegram пушит каждое сообщение POST'ом на `/api/telegram` нашего сервера. Vercel function запускается, обрабатывает, отвечает. Гораздо эффективнее long-polling'a.

Безопасность: webhook верифицируется через `secret_token` header, который Telegram аппендит после `setWebhook`. Без него запрос отклоняется.

Web Crypto API — шифрование credentials

Раздел Credentials хранит логины/пароли с **client-side AES-GCM-256**. Master password никогда не покидает браузер.

Алгоритм:

1. Юзер вводит мастер-пароль.
2. **PBKDF2-SHA256** растягивает его в 256-битный ключ за 600 000 итераций (рекомендация OWASP).
3. **AES-GCM** шифрует каждое поле (`username` , `password` , `notes`) **отдельной IV**. Per-field IVs критически важны: использование одной IV на всех полях позволило бы атакующему сравнивать ciphertexts и извлекать паттерны.
4. На сервер уходят только blobs + IVs.

При расшифровке:

1. PBKDF2 → ключ из мастер-пароля.
2. AES-GCM с per-field IV → plaintext.

Невозможно восстановить, забыв мастер-пароль. Это by design.

Cumulative dependency footprint

@aws-sdk/client-s3	~3 MB	R2 SDK
@dnd-kit/core + sortable	~50 KB	kanban drag
@huggingface/transformers	~5 MB	embeddings (lazy-loaded)
@supabase/ssr + supabase-js	~150 KB	DB client
@upstash/ratelimit + redis	~30 KB	rate limit
fflate	~12 KB	ZIP creation
grammy	~80 KB	bot
next	~30 MB	framework
react / react-dom	~150 KB	
tailwindcss	~5 MB	CSS sweep
web-push	~80 KB	server-side push
zod	~50 KB	validation

Production bundle для самой тяжёлой страницы — около 200 KB JavaScript, remaining ~30 KB первой загрузки + отложенные через `next/dynamic`.

Часть III · Этапы создания

Проект строился волнами. Каждая волна = одна фича end-to-end: миграция БД → DAL → API → UI → тесты → деплой → обновление BACKLOG.

Волна 0 · Базовый прототип

Статический HTML с jQuery-подобным JS, sessionStorage. Это была proof-of-concept для дизайна и UX. Никакой production-готовности.

Волна 1 · Foundation (Next.js + Supabase)

- Next.js 16 проект с App Router.
- Supabase подключён, RLS включена.
- 7 таблиц, 18 RLS-политик, базовый CRUD для entries.

- Auth через magic-link.
- Header + 13 категорий-placeholder'ов.

Волна 2 · Core CRUD + realtime

- Полные CRUD для entries / kanban / credentials.
- Optimistic mutations + supabase Realtime синк между устройствами.
- Edit modal, drag-and-drop kanban.

Волна 3 · Media + credentials

- R2 upload с presigned PUT.
- Browser-side AES-GCM для credentials.
- 18 RLS-политик расширены.

Волна 4 · Telegram bot + cron

- grammY-handler в одном файле.
- `/api/telegram` webhook + secret_token.
- `/api/telegram/digest` cron в 06:00 UTC.

Волна 5 · PWA + edit modal + search

- Service worker (cache-first / SWR / network-first).
- Manifest.json + icons → installable.
- Postgres FTS с tsvector + ILIKE fallback.

Волна 6 · Production deploy + Playwright

- Деплой на Vercel.
- 5 Playwright-спек, реальный Chromium против live URL.
- Backend integration scripts: e2e-credentials, e2e-r2-upload, e2e-telegram.

Волна 7 · Bug audit + рефакторинг

- Прошли по всему коду, нашли несколько мелких багов.
- SUPABASE_SERVICE_ROLE_KEY оказался anon — обновили.
- Fraunces/DM Sans не поддерживали кириллицу — переключились на Mangrove.

Волна 8 · Speed optimization

- Lazy-load heavy modules (KanbanBoard, CredentialsView).
- Service worker для offline.
- Bundle code-splitting.

Волна 9 · Phase 6 — Semantic search

- Migration vector(1536) → vector(384).
- Browser-side e5-small через `@huggingface/transformers`.
- HNSW index + `search_entries_semantic` RPC.
- Reindex button в Settings + backfill flow.

Волна 10 · og: extract + URL auto-fill

- Server-side `lib/og.ts` с SSRF guard'ом.
- AddItemModal автоматически подтягивает title/description/thumbnail.
- Telegram bot для не-YouTube ссылок тоже использует og:.

Волна 11 • Hybrid search RRF

GRIMOIRE VAULT · PROJECT-STORY

- `searchEntriesHybrid` с Reciprocal Rank Fusion.
- Третий режим в `/search`: «Гибрид · RRF».
- Дефолт для семантических запросов.

Волна 12 • 🌀 K Command Palette

- Глобальная палитра, открывается с любой страницы.
- `Empty / live-search / URL-detection` три режима.
- Smart category inference (YouTube / GitHub / Figma / Dribbble).

Волна 13 • Inbox triage

- `entries.triaged_at` колонка + partial-index.
- Per-row + bulk actions (Filed / Move / Delete).
- Header badge с realtime-счётчиком.
- **Поймали critical bug**: `updateEntrySchema = createEntrySchema.partial()` наследовал `.default(...)` значения, и каждый PATCH тихо переписывал `importedVia / metadata / tags / pinned` дефолтами. Регрессия, которая стирала пользовательские теги при любом редактировании. Фикс: явный `z.object({ ... .optional() })` без `.default()`.

Волна 14 • Vim keyboard nav

- `useEntryKeyboardNav` hook.
- `j/k/gg/G/E/P/X/Enter/Esc + ? help-overlay`.
- Игнорируется внутри `input/textarea/contenteditable`.

Волна 15 • Bulk select

- В категориях: `shift+click + tag/pin/move/delete bar`.
- В `/search`: то же самое поверх `search results`.
- `BulkActionsBar` reusable.

Волна 16 • Persistent UI prefs

- `useLocalStorageState` hook (SSR-safe, validate-on-hydrate).
- Search mode + category filter + inbox view persistent.

Волна 17 • Duplicate detection

- `lib/dedup.ts` — URL canonicalization + sha256.
- Tracking-param стрипы (`utm_*`, `fbclid`, etc).
- 409 с deep-link к существующей записи.
- Bot dup-aware reply («🔄 Уже сохранено в ...»).
- Cross-channel: web-form / 🌀 K / bot все идут через тот же `content_hash`.

Волна 18 • Backup story

- `/api/export` (JSON), `/api/export/full` (ZIP с R2 binaries), `/api/import` (cross-account migration).
- Settings panels: `ExportVault`, `ImportVault`.
- Dedup при re-import через тот же `content_hash`.

Волна 19 • Owner-only ops layer

8 / 13

- `lib/admin.ts` с `requireOwner()` + `OWNER_EMAIL` env.

- AdminStats panel в Settings.
- /admin/health page (5-tile probe).
- Danger zone (двухстадийный wipe).

Волна 20 · Observability

- Structured JSON logs.
- Per-request UUID + `X-Request-Id` header на ошибках.
- Request timing (`durationMs`) на каждом запросе.

Волна 21 · Rate limiting + Web Push + Shared vaults

Финальный sprint: три «open roadmap» пункта закрыты в один заход.

- **Rate limiting:** Upstash Redis (sliding window) с in-memory fallback.
- **Web Push:** VAPID keys, push_subscriptions table, Service Worker push handler, Settings toggle, bot integration.
- **Shared vaults:** vaults / vault_members / vault_invites schema, RLS-rewrite на entries, VaultPicker в Header, Settings → Vaults panel, /invite/[code] landing page. По дороге поймали ещё два RLS-бара (recursive policy и trigger без SECURITY DEFINER).

Часть IV · Что для простого юзера, что для продвинутого

Простому юзеру нужно

1. **Категории + теги** — фундаментальный UX. 13 «комнат», теги для нюансов.
2. **Telegram-бот** — capture с любого устройства за секунды.
3. **Inbox триаж** — ритуал «к нулю» каждый день.
4. **⌘K** — быстрая навигация и сохранение URL.
5. **Поиск** — базовый FTS закрывает 90% случаев.
6. **PWA + push** — на телефоне ставится один раз, дальше как нативное приложение.
7. **Credentials** — если есть, что прятать. Если нет — игнорь раздел.

Продвинутому интересны

1. **Семантический поиск** — описательные запросы вместо ключевых слов.
2. **Hybrid mode (RRF)** — лучше для сложных запросов.
3. **Bulk operations** — массовое теггирование, миграции между категориями.
4. **Vim-keyboard nav** — если ты кодер и любишь руки на клавиатуре.
5. **Shared vaults** — для семьи / команды.
6. **Export/Import** — миграция между аккаунтами, бэкап-дисциплина.
7. **Web Push** — пуши о новых записях.

Только для разработчика / админа

1. **AdminStats** — KPI, embedding coverage, R2-разбивка.
2. **/admin/health** — пробит зависимостей.
3. **Danger zone wipe** — для рестарта с чистого листа.
4. **Reindex embeddings** — батч-инструмент.
5. **Vercel Logs Explorer** — дебаг через `requestId`.
6. **Upstash Redis** — для production-grade rate limiting.

ФИЧА	NOTION	OBSIDIAN	LOGSEQ	GRIMOIRE VAULT
Облачная синхронизация	✅ платно	⚠️ платно	❌ self-host	✅ бесплатно
Open source	❌	⚠️ частично	✅	✅
Multi-device	✅	⚠️ через sync	⚠️ через sync	✅
Мобильное приложение	✅	✅ медленное	⚠️	✅ PWA
Telegram-бот	❌	❌	❌	✅
Семантический поиск	⚠️ платно (AI)	❌	❌	✅ бесплатно (on-device)
Зашифрованные пароли	❌	❌	❌	✅ AES-GCM
Drag-and-drop kanban	✅	⚠️ через плагин	✅	✅
Shared vaults	✅ платно	❌	❌	✅ бесплатно
Полный экспорт	⚠️ медленный	✅ (.md)	✅ (.md)	✅ JSON + ZIP с binaries
Import обратно	❌	❌	❌	✅
Web Push	❌	❌	❌	✅
Кастомизация	⚠️ template'ы	✅ плагины	✅ плагины	⚠️ только форк
Цена для personal	\$0-10/mo	\$0 (sync \$96/yr)	\$0	\$0

Где Grimoire Vault выигрывает: capture-friction (бот за 2 секунды), семантический поиск без платных API, шифрование паролей рядом с заметками, экспорт+импорт в один клик.

Где проигрывает: нет плагинов / расширений (только форк), нет встроенного редактора длинных документов (записи — короткие entries, не статьи), нет blocks-семантики (как в Notion), нет outlining (как в Logseq).

Когда выбирать Grimoire Vault: ты хочешь быструю личную базу закладок + заметок с фокусом на capture, не на длинное письмо.

Когда не выбирать: если ты пишешь книгу, нужен outline-режим, или уже жил в Obsidian и зависим от плагинов.

Часть VI · Что важно понять о работе с базами данных

Чтобы не запутаться, открыв этот проект:

1. SQL ≠ NoSQL. Postgres — это SQL.

Каждая запись (entry, vault, credential) живёт в **таблице** со строгим набором столбцов. Это не «document store» как MongoDB. Если хочется добавить поле — нужна **миграция** (см. `supabase/migrations/`).

2. RLS — авторизация на уровне БД, не приложения

Когда ты делаешь `supabase.from("entries").select(...)`, под капотом PostgREST проверяет RLS-политики и фильтрует строки. **Даже если скомпрометирован аноп-ключ, нельзя прочитать чужие записи.** Это лучше, чем ручные `WHERE user_id = ...` в коде, потому что забыть такой WHERE — security bug, а забыть про RLS невозможно (Postgres вернёт пустой результат).

3. Транзакции — атомарность

`supabase-js` не позволяет открыть multi-statement transaction явно (это ограничение PostgREST). Если нужна атомарность — пиши SQL function (RPC), вызывай через `.rpc(...)`. Так сделано для `search_entries_semantic` и `count_entries_per_category`.

4. Индексы — ключ к performance

Без индекса `WHERE category_id = 'misc'` в таблице из 100k строк работает sequential scan (медленно). С `entries_user_cat_idx` — index scan (микросекунды).

В нашем проекте:

- Btree-индексы на FK / часто фильтруемых столбцах.
- **Partial indexes** — индексируют только часть строк (`WHERE imported_via = 'bot' AND triaged_at IS NULL` — для inbox). Маленькие, быстрые.
- **HNSW** для cosine similarity (vector(384)).
- **GIN** для tsvector и tags.

5. Триггеры — побочные эффекты на INSERT/UPDATE

В нашей БД четыре trigger function:

- `entries_update_search_tsv` — пересчитывает tsvector при изменении text-полей.
- `entries_set_triaged_default` — для не-bot записей сразу ставит `triaged_at = now()`.
- `vaults_seed_owner` — добавляет создателя в `vault_members`.

6. RPCs — серверные функции

`search_entries_semantic(query_embedding, ...)` — SQL function, вызывается из клиента через `.rpc()`. Это способ упаковать сложную логику в один round-trip и переиспользовать через RLS.

7. Realtime — websocket поверх WAL

Supabase Realtime читает Postgres WAL (write-ahead log) и бродкастит INSERT/UPDATE/DELETE подписчикам через websocket. **RLS применяется к каждому событию** — пользователь видит только свои changes.

Часть VII · На что обратить внимание в первую очередь

Если ты только что зашёл в репозиторий и хочешь быстро понять, что происходит:

1. Прочти три файла в корне

GRIMOIRE VAULT · PROJECT-STORY

```
README.md      # обзор проекта, deploy guide
docs/USER.md   # как пользоваться
docs/DEVELOPER.md # как развивать
```

2. Посмотри на 8 миграций — это контракт БД

```
supabase/migrations/
├─ 20260504000000_initial_schema.sql      ← начини отсюда
├─ 20260504010000_credentials_per_field_iv.sql
├─ 20260504020000_count_entries_per_category.sql
├─ 20260504030000_embedding_384.sql
├─ 20260504040000_entries_triaged_at.sql
├─ 20260504050000_dedup_index_unpartial.sql
├─ 20260504060000_push_subscriptions.sql
└─ 20260504070000_shared_vaults.sql
```

Прочитанные подряд они показывают эволюцию модели данных.

3. Открой `lib/data/entries.ts` — это сердце проекта

Самая центральная сущность — entry. Все остальные части (UI, API, search, dedup, triage, vaults) вращаются вокруг неё.

4. Изучи `lib/api-helpers.ts` → `withErrorHandler`

Понимая этот wrapper, ты понимаешь, как **каждый** API-route ведёт себя: auth, rate limit, error mapping, logging, request ID.

5. Запусти локально

```
git clone ...
cd grimoire-vault
npm install
cp .env.example .env.local
# заполни env vars
npm run dev
```

Прокликай UI в браузере — особенно 🔍, search, /inbox, /settings. Лучше один час использования, чем пять часов чтения.

6. Открой Vercel Logs Explorer (если есть доступ)

Посмотри, как структурированные логи выглядят. Filter `level:warn`, `level:error`. Это инструмент для daily ops.

Часть VIII · Долгий взгляд

Этот проект — **personal**. Не SaaS-стартап, не open-source-движение (хотя репозиторий публичный). Цель — иметь рабочий «второй мозг», который **не зависит от чужой компании, чужого монетизационного roadmap'a, чужих ограничений на бесплатном плане**.

Stack выбран так, чтобы:

1. **Бесплатные tier’ы** покрывали персональное использование на годы (Supabase 500 MB, Vercel hobby, R2 10 GB, Telegram бесплатно).

GRIMOIRE VAULT · PROJECT-STORY

2. **Замена любого компонента** была реальной за день: Supabase → Postgres + own auth, R2 → S3, Vercel → любой Next-host.

3. **Все данные** были легко вынимаемы (JSON/ZIP экспорт).

Если завтра Vercel закроют, Supabase купят, R2 поднимет цены — мы не заперты. Один Full export в облако, один импорт в новое место — и работаем дальше.

Это и есть главная идея проекта.

Crafted A.D. MMXXVI · Set in Fraunces, Manrope & JetBrains Mono.